MapReduce Primer

Overview

Teaching: 40 min
Exercises: 0 min

Questions
- What is MapReduce and how does it work?

Objectives
- Understand the principles behind MapReduce.
- Learn why MapReduce is usefule for processing large data sets.

MapReduce is a software framework for processing large data sets in a distributed fashion over a several machines. The core idea behind MapReduce is mapping your data set into a collection of (key, value) pairs, and then reducing over all pairs with the same key.

The overall concept is simple, but is actually quite expressive when you consider that:

- Almost all data can be mapped into (key, value) pairs somehow, and
- Keys and values may be of any type: strings, integers, dummy types, and, of course, (key, value) pairs themselves

The canonical MapReduce use case is counting word frequencies in a large text, but some other examples of what you can do in the MapReduce framework include:

- Distributed sort
- Distributed search
- Web-link graph traversal
- Machine learning

Counting the number of occurrences of words in a text is sometimes considered as the "Hello world!" equivalent of MapReduce. A classical way to write such a program is presented in the python script below. The script is very simple. It parses the file from which it extracts and counts words and stores the result in a dictionary that uses words as keys and the number of occurrences as values.

First, download Moby Dick, the novel by Herman Melville and move the `pg2701.txt` file to your current directory. Next, create a Python program called `word_count.py` using the following code:

```
import re

# remove any non-words and split lines into separate words
# finally, convert all words to lowercase
def splitter(line):
    line = re.sub(r'^\W+|\W+$', '', line)
    return map(str.lower, re.split(r'\W+', line))

sums = {}
try:
    in_file = open('pg2701.txt', 'r')

    for line in in_file:
        for word in splitter(line):
            word = word.lower()
            sums[word] = sums.get(word, 0) + 1

    in_file.close()

except IOError:
    print("error performing file operation")
else:
    M = max(sums.iterkeys(), key=lambda k: sums[k])
    print("max: %s = %d" % (M, sums[M]))
```

When you run this program, you should see the following output:

```
max: the = 14620
```

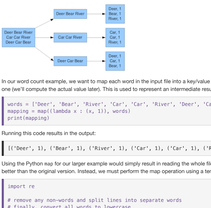
The main problem with this approach comes from the fact that it requires the use of a dictionary, i.e., a central data structure used to progressively build and store all the intermediate results until the final result is reached.

Since the code we use can only run on a single processor, the best we can expect is that the time necessary to process a given text will be proportional to the size of the text (i.e., the number of words processed per second is constant).

In reality though, the performance degrades as the size of the dictionary grows. As shown on the diagram below, the number of words processed per second diminishes when the size of the dictionary reaches the size of the processor data cache (note that if the cache is structured in several layers of different speeds, the processing speed will decrease each time the dictionary reaches the size of a layer).

An even larger decrease in processing speed will occur when the dictionary reaches the size of the computer's Random Access Memory (RAM).

Eventually, if the dictionary continues to grow, it will exceed the capacity of the *swap space* and an exception will be raised.

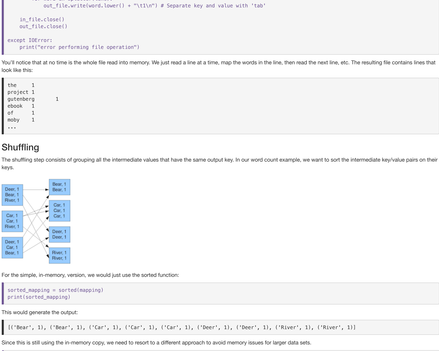


The MapReduce aproach

The main advantage of the MapReduce approach is that it does not require a central data structure so the memory issues that occur with the simplistic approch are avoided.

MapReduce consists of 3 steps:

- A mapping step that produces intermediate results and associates them with an output key;
- A shuffling step that groups intermediate results associated with the same output key; and
- A reducing step that processes groups of intermediate results with the same output key.



Mapping

The mapping step is very simple. The idea is to apply a function to each element of a list and collect the result. This is essentially the same as the Python `map` method that takes a function and sequence of input values and returns a sequence of values that have had the function applied to them.

In our word count example, we want to map each word in the input file into a key/value pair containing the word as key and the number of occurances as the value, where the value is one (we'll compute the actual value later). This is used to represent an intermediate result that says: "this word occurs one time". It is equivalent to the following:

```
words = ['Deer', 'Bear', 'River', 'Car', 'Car', 'River', 'Deer', 'Car', 'Bear']
mapping = map(lambda x : (x, 1), words)
print(mapping)
```

Running this code results in the output:

```
[('Deer', 1), ('Bear', 1), ('River', 1), ('Car', 1), ('Car', 1), ('River', 1), ('Deer', 1), ('Car', 1), ('Bear', 1)]
```

Using the Python `map` for our larger example would simply result in reading the whole file into memory before we can perform the map operation, and unfortunately this would be no better than the original version. Instead, we must perform the map operation using a temporary file (that we will use later), as follows:

```
import re

# remove any non-words and split lines into separate words
# finally, convert all words to lowercase
def splitter(line):
    line = re.sub(r'^\W+|\W+$', '', line)
    return map(str.lower, re.split(r'\W+', line))

input_file = 'pg2701.txt'
map_file = 'pg2701.txt.map'

# Implement our mapping function
import re
sums = {}
try:
    in_file = open(input_file, 'r')
    out_file = open(map_file, 'w')

    for line in in_file:
        for word in splitter(line):
            out_file.write(word.lower() + "\t1\n") # Separate key and value with 'tab'

    in_file.close()
    out_file.close()

except IOError:
    print("error performing file operation")
```

You'll notice that at no time is the whole file read into memory. We just read a line at a time, map the words in the line, then read the next line, etc. The resulting file contains lines that look like this:

```
the     1
project 1
gutenberg       1
ebook   1
of      1
moby    1
...
```

Shuffling

The shuffling step consists of grouping all the intermediate values that have the same output key. In our word count example, we want to sort the intermediate key/value pairs on their keys.

For the simple, in-memory, version, we would just use the sorted function:

```
sorted_mapping = sorted(mapping)
print(sorted_mapping)
```

This would generate the output:

```
[('Bear', 1), ('Bear', 1), ('Car', 1), ('Car', 1), ('Car', 1), ('Deer', 1), ('Deer', 1), ('River', 1), ('River', 1)]
```

Since this is still using the in-memory copy, we need to resort to a different approach to avoid memory issues for larger data sets.

```
map_file = 'pg2701.txt.map'
sorted_map_file = 'pg2701.txt.map.sorted'

def build_index(filename):
    index = []
    f = open(filename)
    while True:
        offset = f.tell()
        line = f.readline()
        if not line:
            break
        length = len(line)
        col = line.split('\t')[0].strip()
        index.append((col, offset, length))
    f.close()
    index.sort()
    return index

try:
    index = build_index(map_file)
    in_file = open(map_file, 'r')
    out_file = open(sorted_map_file, 'w')
    for col, offset, length in index:
        in_file.seek(offset)
        out_file.write(in_file.read(length))
    in_file.close()
    out_file.close()
except IOError:
    print("error performing file operation")
```

Reducing

For the reduction step, we just need to count the number of values with the same key. Now that the different values are ordered by keys (i.e., the different words are listed in alphabetic order), it becomes easy to count the number of times they occur by summing values as long as they have the same key.

For the simple, in-memory, version, we can use lambda functions like this:

```
from itertools import groupby

# 1. Group by key yielding (key, grouper)
# 2. For each pair, yield (key, reduce(func, last element of each grouper))
grouper = groupby(sorted_mapping, lambda p:p[0])
print(map(lambda t: (t[0], reduce(lambda x, y: x + y, map(lambda p:p[1], t[1]))), grouper))
```

The resulting output is:

```
[('Bear', 2), ('Car', 3), ('Deer', 2), ('River', 2)]
```

For our sorted mapping file, it's also straight forward. We just read each key/value pair and continue to count until we find a different key. We just print out the value, then reset the values for the next key.

```
previous = None
M = [None, 0]

def checkmax(key, sum):
    global M, M
    if M[1] < sum:
        M[1] = sum
        M[0] = key

try:
    in_file = open(sorted_map_file, 'r')
    for line in in_file:
        key, value = line.split('\t')

        if key != previous:
            if previous is not None:
                checkmax(previous, sum)
            previous = key
            sum = 0

        sum += int(value)

    checkmax(previous, sum)
    in_file.close()
except IOError:
    print("error performing file operation")

print("max: %s = %d" % (M[0], M[1]))
```

Running this prints the same result we saw before:

```
max: the = 14620
```

Although these three steps seem like a complicated way to achieve the same result, there are a few key differences:

- In each of the three steps, the entire contents of the file never had to be held in memory. This means that the program is not affected by the same caching issues as the simple version.
- The mapping function can be be split into many independent parallel tasks, each generating separate files.
- The shuffling and reducing functions can also be split into many independent parallel tasks, with the final result being written to an output file.

The fact that the MapReduce algorithm can be parallelized easily and efficiently means that it is ideally suited for applications on very large data sets, as well as were resiliance is required.

MapReduce is clearly not a general-purpose framework for all forms of parallel programming. Rather, it is designed specifically for problems that can be broken up into the the map-reduce paradigm. Perhaps surprisingly, there are a lot of data analysis tasks that fit nicely into this model. While MapReduce is heavily used within Google, it also found use in companies such as Yahoo, Facebook, and Amazon.

The original, and proprietary, implementation was done by Google. It is used internally for a large number of Google services. The *Apache* Hadoop project built a clone to specs defined by Google. Amazon, in turn, uses Hadoop MapReduce running on their EC2 (elastic cloud) computing-on-demand service to offer the Amazon Elastic MapReduce service.

Key Points

- MapReduce is a software framework for processing large data sets in a distributed fashion.
- A data set is mapped into a collection of (key value) pairs.
- The (key, value) pairs can be manipulated (e.g. by sorting).
- The result is a reduction over all pairs with the same key.